

# UE25CS243A – Reliable UDP File Transfer

Project Guide • Checklist • Contribution Rules

Team: Sameer • Siri • Saraswath • Ashvath

## 1. Project Overview

We are implementing **Project 4 – Reliable File Transfer** using UDP sockets. UDP has no reliability, so we build sequence numbers, ACKs, retransmission and timeouts ourselves in pure C. The SDN controller (Ryu) will change network paths during transfer to test recovery. Everything is implemented from scratch.

## 2. Repository Folder Structure

```
UE25CS243A-Reliable-UDP-File-Transfer/
■■■ README.md
■■■ docs/
    ■■■ Project_Approach.pdf
    ■■■ Project_Guide_and_Checklist.pdf ← this file
■■■ src/
    ■■■ client.c
    ■■■ server.c
    ■■■ packet.h
    ■■■ Makefile
■■■ controller/
    ■■■ ryu_path_switch.py
■■■ mininet/
    ■■■ topology.py
■■■ scripts/
    ■■■ test_transfer.sh
■■■ results/                                     (screenshots, logs, numbers)
```

## 3. Current Status Checklist

Update this section whenever something is finished. Use [x] for done, [ ] for pending.

| Status | Task                                              | Owner / Notes       |
|--------|---------------------------------------------------|---------------------|
| [ ]    | Basic UDP client + server (send/receive file)     | Sameer – Start here |
| [ ]    | Add sequence numbers + Stop-and-Wait              |                     |
| [ ]    | Upgrade to Selective Repeat (sliding window)      |                     |
| [ ]    | Checksum + duplicate detection + FIN              |                     |
| [ ]    | File integrity check (size + hash)                |                     |
| [ ]    | Makefile + clean compile instructions             |                     |
| [ ]    | Mininet topology with 2 alternate paths           |                     |
| [ ]    | Ryu controller – install Path A flows             |                     |
| [ ]    | Ryu controller – switch to Path B mid-transfer    |                     |
| [ ]    | End-to-end test: transfer survives path change    |                     |
| [ ]    | Performance numbers (time, retransmits, recovery) |                     |
| [ ]    | Final README + documentation                      |                     |
| [ ]    | GitHub repo ready for demo                        |                     |

## 4. How to Contribute

Only people who actually write code will remain as collaborators.

### Step-by-step workflow:

1. Clone the repository
2. Create your own branch (never work directly on main)
3. Make your changes

4. Commit with a clear message
5. Push your branch
6. Open a Pull Request (PR) on GitHub

**Useful Git commands:**

```
git clone <repo-url>
git checkout -b feature/basic-udp-transfer
git add .
git commit -m "Add basic UDP client and server"
git push -u origin feature/basic-udp-transfer
```

## 5. How to Make a Pull Request (PR)

1. Go to the repository on GitHub.
2. You will see a yellow banner: “**Compare & pull request**” — click it.
3. Write a short title (example: “Add basic UDP file transfer”).
4. In the description write what you did and how to test it.
5. Click **Create Pull Request**.
6. Wait for review. Do not merge your own PR unless you are the main owner.

Never push directly to **main**. Always use a feature branch + Pull Request.

## 6. Coding Rules

- Language for application: **pure C** (POSIX sockets only)
- No external reliability libraries
- Controller: Ryu (Python) is allowed
- Use clear variable names and comments
- Every major feature should be in its own branch
- Test on localhost first before touching Mininet

## 7. Team & Suggested Division of Work

- **Sameer** (Repo Owner): Core C client + server + reliability (Selective Repeat)
- **Siri** (Active Collaborator): Mininet topology + basic connectivity testing + help with integration
- **Saraswath & Ashvath**: Can join any pending task once they start contributing

Anyone can help with documentation and performance measurements. Update the checklist when you finish a task.

---

Update the checklist regularly. Keep the main branch always in a working state. Discuss in the team group before making big changes.